

ATTENTION RESIDUALS

<https://arxiv.org/pdf/2603.15031>

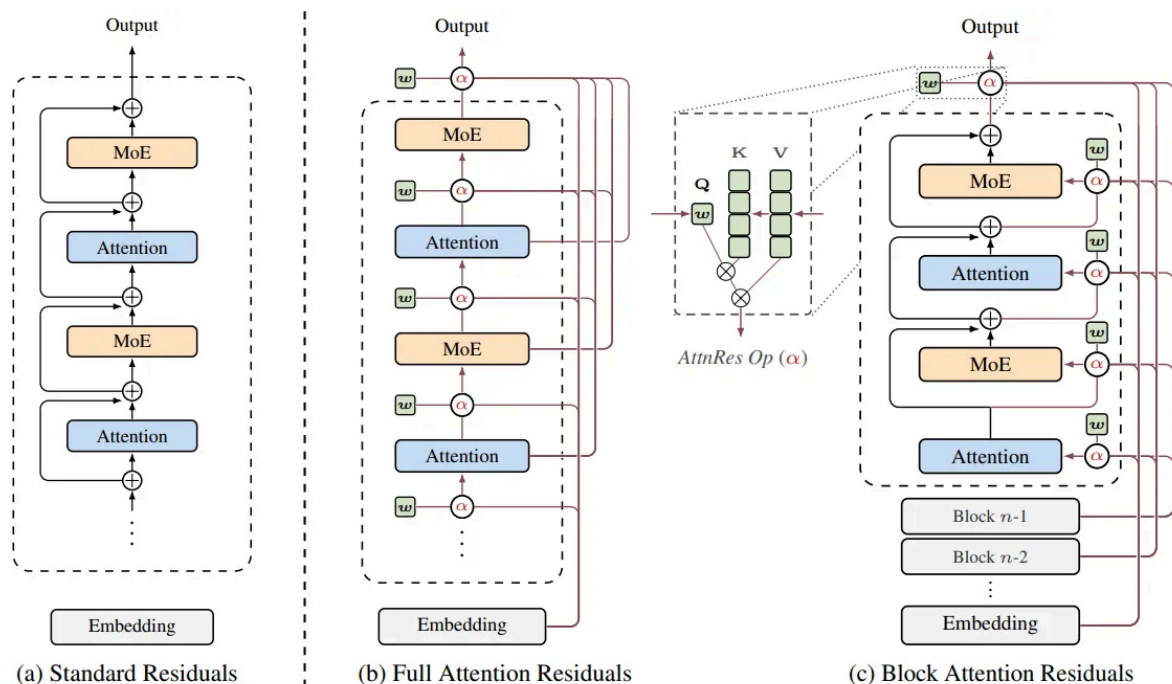


Figure 1: Overview of Attention Residuals. **(a)** Standard Residuals: standard residual connections with uniform additive accumulation. **(b)** Full AttnRes: each layer selectively aggregates all previous layer outputs via learned attention weights. **(c)** Block AttnRes: layers are grouped into blocks, reducing memory from $O(Ld)$ to $O(Nd)$.

1. 先看 Transformer 里的 Residual 到底是什么

标准 Transformer block:

```
graph TD
    x1[x_1] --> A[Attention]
    A --> plus1[+]
    x1 --> plus1
    plus1 --> v1[v]
    v1 --> LN[LayerNorm]
    LN --> FFN[FFN]
    FFN --> plus2[+]
    x1 --> plus2
    plus2 --> v2[v]
    v2 --> x1plus1[x_{l+1}]
```

数学：

$$x_{l+1} = x_l + F(x_l)$$

其中：

- x_l ：第 l 层输入
- $F(x_l)$ ：这一层学习的新信息

这个设计来自 ResNet。

意思：

不要让每一层重新生成全部信息，只学习“变化量”。

2. 但是 LLM 越来越深，Residual 出问题了

假设 4 层：

第1层：

$$x_1 = x_0 + h_1$$

第2层：

$$x_2 = x_0 + h_1 + h_2$$

第3层：

$$x_3 = x_0 + h_1 + h_2 + h_3$$

展开：

最终：

$$x_L = x_0 + \sum_{i=1}^L h_i$$

也就是说：

所有层的信息全部堆起来。

例如 100 层：

```
Layer100:

x0
+
Layer1信息
+
Layer2信息
+
Layer3信息
...
+
Layer99信息
```

问题：

每一层权重都是 1

也就是：

$$x_L = 1 * x_0 + 1 * h_1 + 1 * h_2 \dots + 1 * h_L$$

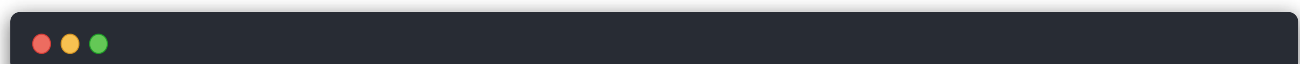
没有选择。

3. 类比人的记忆

假设你读一本书。

普通 residual：

你的笔记：



第一页重点
第二页重点
第三页重点
...
第100页重点

全部堆进去。

问题：

第一页的信息：

● ● ●
苹果是什么

到了最后：

被99页内容淹没。

这就是论文说的：

residual dilution (残差稀释)

随着深度增加，每层贡献越来越被稀释。 ([ResearchGate][3])

4. Attention Residual 的核心思想

论文想：

不要：

$$x_L = x_0 + \sum h_i$$

而是：

让第 L 层：

"选择性读取以前哪些层的信息"

变成：

$$x_L = \sum_{i=0}^{L-1} \alpha_i h_i + F(x_{L-1})$$

其中：

$$\alpha_i = softmax(q_L k_i)$$

也就是说：

给每个历史 layer 一个 attention score。

5. 一个具体例子

假设 6 层 Transformer：

```
Layer output:
```

```
h0  输入
h1  语法信息
h2  世界知识
h3  推理过程
h4  数学步骤
h5  当前状态
```

传统 residual：

全部加：

```
h0+h1+h2+h3+h4+h5
```

权重：

```
h0 1
h1 1
h2 1
h3 1
h4 1
h5 1
```

Attention Residual:

当前 token 计算:



我要回答数学问题

attention:



h0 输入	0.05
h1 语法	0.05
h2 知识	0.10
h3 推理	0.40
h4 数学	0.35
h5 当前	0.05

于是:

$$x = 0.4h_3 + 0.35h_4 + 0.1h_2 \dots$$

模型自己决定:

当前任务需要哪些深度的信息。

6. 为什么叫 Attention Residual?

因为它把:

Transformer 中 token 维度 attention:



```
token attention
```

```
token1 ---> token2
```

```
token3 ---> token4
```

扩展到了:

depth 维度 attention

以前:



```
Layer 1
```

```
|
```

```
Layer 2
```

```
|
```

```
Layer 3
```

```
|
```

```
Layer 4
```

只能顺序累加。

现在:



```
Layer1
```

```
↑
```

```
Layer4 ---> Attention
```

```
↓
```

```
Layer2
```

```
↓
```

```
Layer3
```

第 L 层可以看所有之前层。

所以叫:

Attention over depth

7. 和 DenseNet 的区别

很多人第一反应：

"这不是 DenseNet 吗？"

有点像。

DenseNet:

```
layer4:
  concat(
    layer1,
    layer2,
    layer3
  )
```

但是：

DenseNet:

固定全部使用。

AttnRes:

学习选择：

```
layer4:

0.1 layer1
0.7 layer2
0.2 layer3
```

8. 计算公式

普通:

$$x_l = x_{l-1} + f_l(x_{l-1})$$

Attention Residual:

保存:

$$H_l = [h_0, h_1, \dots, h_{l-1}]$$

然后:

Query:

$$q_l = W_q x_l$$

Key:

$$k_i = W_k h_i$$

Value:

$$v_i = W_v h_i$$

Attention:

$$a_i = \text{softmax}(q_l k_i^T)$$

输出:

$$r_l = \sum_i a_i v_i$$

最终:

$$x_l = r_l + f_l(x_l)$$

9. 最大的问题: 计算量

如果 100 层:

第100层：

看：

99个历史层。

如果每层都看：

复杂度：

$$O(L^2)$$

其中 L 是层数。

所以论文提出：

Block Attention Residual

不要每层看所有层：

例如：

```
Block1:
Layer1-8

Block2:
Layer9-16

Block3:
Layer17-24
```

块内部：

普通 residual。

块之间：

Attention。

类似：



降低开销。([Papers with Code Search][4])

10. 它和 Kimi Linear 的关系

这个很关键。

Kimi 的路线：



它们解决的问题不同：

方法	优化什么
Linear Attention	token 长度 $O(n^2)$
Delta Attention	KV 状态更新
KDA	attention memory
Attention Residual	layer depth 信息融合

一个是在：



token 维度

压缩。

一个是在：



depth 维度

重新设计。

11. 最直观总结

普通 Transformer:

每一层把自己的信息丢进一个大桶，最后大家混在一起。

Attention Residual:

每一层有一个智能检索器，需要信息时，从过去层里面挑重点。

所以：

Residual Connection:



过去所有信息全部相加



Attention Residual:



过去所有层作为memory，用attention读取

([alphaXiv][2])

这个论文其实和你之前研究的 **KDA / Delta Attention** 有一个非常有意思的对应关系：

- KDA：解决 **时间维度 token memory 如何更新**
- AttnRes：解决 **深度维度 layer memory 如何读取**

一个是在序列方向做 memory，一个是在网络深度方向做 memory。